

Development Prompt Playbook

VITALS-Edge — build the whole system, prompt by prompt

A structured, copy-paste-ready set of engineering prompts that take VITALS-Edge from an empty repository to a running, demoable edge-AI predictive-maintenance system. Feed them to Claude Code, Cursor, or a chat assistant in order.

Companion to the VITALS-Edge technical dossier

Team Algorithm404 · Dept. of AI & Data Science, Sri Eshwar College of Engineering

How to Use This Playbook

Each numbered card is a self-contained prompt. Paste it into your AI coding assistant more or less verbatim, tweak the specifics in brackets, and review the output before moving on. The cards are ordered so each one builds on the files created by the previous ones.

The one rule that matters

Always load Prompt 0 first. It is the master context that tells the assistant what VITALS-Edge is, the fixed tech stack, and the non-negotiable design rules (edge-first, hardware-optional demo). In Claude Code, put it in `CLAUDE.md`; in Cursor, put it in `.cursorrules`; in a chat, paste it once at the top of the session.

Prompt-quality principles baked into every card

- **Role + context first** — the assistant is told who it is and what it's building before the task.
- **One clear task per prompt** — narrow enough that the output is reviewable in one sitting.
- **Explicit constraints** — stack, conventions and limits stated, so you don't get exotic dependencies.
- **Named deliverables** — exact files/functions expected, so nothing is left vague.
- **Acceptance criteria** — a checklist the code must satisfy; also your review checklist.

Working loop for each card

- Run the prompt → read the plan the assistant proposes → let it write the code.
- Run it / run its tests. If it breaks, use the **Debug** meta-prompt (Prompt 23) with the error pasted in.
- When it passes the acceptance criteria, commit, then move to the next card.

0 Master Context (load this first)

This is the foundation prompt. Everything else assumes the assistant has already read it.

PROMPT 0 — Master project context

→ [CLAUDE.md](#) / [.cursorsrules](#) / [top of chat](#)

ROLE: You are a senior ML + backend engineer pair-building "VITALS-Edge" with me for the Tata InnoVent-27 hackathon (theme: AI at the Edge; category 3.2.1.3, Edge AI for Vehicle Health & Predictive Maintenance). Judging rewards technical feasibility and a working live demo.

GOAL: Detect incipient engine / rotating-machinery faults ON THE DEVICE in under 2 seconds, estimate Remaining Useful Life (RUL) in real time, and send only <1 KB alert packets to a thin cloud dashboard. All inference must run at the edge and keep working with no network.

FIXED TECH STACK: Python 3.11; PyTorch (training); ONNX Runtime / TensorRT (edge inference); FastAPI + Uvicorn (edge service); paho-mqtt + Mosquitto (messaging); InfluxDB or SQLite (storage); Streamlit (dashboard). Target edge HW: NVIDIA Jetson Orin Nano Super, but EVERYTHING must also run on a plain laptop CPU.

DATASETS: NASA C-MAPSS (turbofan RUL), CWRU bearing (vibration faults), AI4I 2020 (tabular).

TOP DESIGN RULE - HARDWARE-OPTIONAL DEMO: We must be able to demo the full pipeline WITHOUT a real engine, by replaying recorded dataset runs as a real-time sensor stream into the exact same edge service used in the field. Never assume live hardware exists.

CONVENTIONS: type hints everywhere; pydantic models for every API and MQTT payload; config via pydantic-settings + .env (no hardcoded paths); structured logging; pytest unit tests; ONE shared preprocessing module imported by both training and serving (no train/serve skew); small composable modules; keep the edge model compact enough for INT8 quantization.

HOW TO RESPOND: When I give a task, first state the file layout / plan if new, then give complete runnable code with imports, then say exactly how to test it. Ask a clarifying question only when a choice is hard to reverse. Prefer standard, well-supported libraries.

A Foundation

PROMPT 1 — Repository scaffold & tooling

→ Claude Code (agentic)

CONTEXT: Fresh repo for VITALS-Edge. Set up the skeleton only - no model logic yet.

TASK: Create a clean Python monorepo layout with these packages: `vitals_common` (shared schemas, config, preprocessing), `vitals_data` (dataset loaders + replay harness), `vitals_train` (model training + export), `vitals_edge` (FastAPI edge service, inference, decision engine, alerts), `vitals_cloud` (mqtt subscriber, storage, digital twin, dashboard), and `tests/`.

REQUIREMENTS: `pyproject.toml` (or `requirements.txt`) pinning the fixed stack; `.env.example`; Makefile with targets (`install`, `lint`, `test`, `run-edge`, `run-dashboard`, `replay`); pre-commit config (`ruff + black`); a README skeleton with the architecture summary; `.gitignore` for Python + data.

DELIVERABLES: the full directory tree, all config files, empty `__init__.py` files, and a one-line docstring in each package saying its responsibility.

ACCEPTANCE: ``make install`` works; ``make lint`` and ``make test`` run (even with zero tests); importing every package succeeds.

PROMPT 2 — Config + shared payload schemas

→ chat or Claude Code

CONTEXT: Build the contracts that both edge and cloud depend on, in `vitals_common`.

TASK: Define (a) a `pydantic-settings` `Settings` class reading all config from `.env` (paths, MQTT host/port/topics, model paths, window size, thresholds, sample rate); and (b) the `pydantic` models exchanged across the system.

REQUIREMENTS: models needed - `SensorWindow` (`asset_id`, `timestamp`, sensor arrays / features); `Prediction` (`anomaly_score`, `rul_estimate`, `fault_class`, `confidence`); `HealthEvent` (`asset_id`, `state`, `prediction`, `timestamp`) where `state` is an enum `HEALTHY/WATCH/WARNING/CRITICAL`; `Alert` (compact, serialises to <1 KB JSON). Add validators for shapes/ranges.

DELIVERABLES: `vitals_common/config.py`, `vitals_common/schemas.py`, and a small `pytest` proving each model round-trips to/from JSON and the `Alert` stays under 1 KB.

ACCEPTANCE: all schemas import and validate; `Alert` JSON < 1024 bytes for a realistic payload.

B Data Layer

PROMPT 3 — Dataset loaders

→ chat or Claude Code

CONTEXT: `vitals_data` needs to load the three public datasets into a common format.

TASK: Write loaders for NASA C-MAPSS (FD001-FD004), CWRU bearing vibration, and AI4I 2020. Each returns clean `pandas` / `numpy` structures plus a per-unit "cycle" and a ground-truth RUL where available. Include a short docstring with the exact download URL and expected folder layout.

REQUIREMENTS: no hardcoded absolute paths (use `Settings`); graceful error if files are missing, with the download instructions in the message; cache parsed data to `Parquet`.

DELIVERABLES: `vitals_data/loaders.py` with `load_cmapss(subset)`, `load_cwru()`, `load_ai4i()`; a `tests` file using a tiny bundled sample or synthetic stand-in so tests run without the full download.

ACCEPTANCE: loaders return expected shapes; RUL column present for C-MAPSS; tests pass offline.

PROMPT 4 — Shared preprocessing module

→ [chat](#) or [Claude Code](#)

CONTEXT: The single most important anti-bug module - training and the edge MUST use identical preprocessing. Lives in `vitals_common` so both import it.

TASK: Implement windowing (sliding windows of size `N` with stride `S`), per-channel scaling (fit on train, saved as an artifact and reloaded at serve time), and vibration feature extraction (FFT magnitude + envelope spectrum + RMS/kurtosis) behind one function `make_features(window)`.

REQUIREMENTS: the fitted scaler is saved/loaded (`joblib`); no leakage (fit only on training data); pure functions, no global state; fully type-hinted.

DELIVERABLES: `vitals_common/preprocess.py` + tests asserting a saved scaler reproduces identical output on the same input.

ACCEPTANCE: identical feature vector from training path and serving path for the same raw window.

PROMPT 5 — Replay harness (the demo enabler)

→ [Claude Code](#)

CONTEXT: This is what lets us demo with no engine. It streams a recorded run at real-time speed.

TASK: Build a replay harness that reads a dataset run (or a raw CSV) row-by-row and publishes each window as a `SensorWindow`, either to an MQTT topic OR via a direct in-process callback, at a configurable rate (e.g. 1x / 5x / 50x real time).

REQUIREMENTS: deterministic with a seed; a `--fault` flag that switches from a healthy run to a run-to-failure run mid-stream (for the live "fault develops" moment); clean start/stop; CLI via `argparse` or `typer`.

DELIVERABLES: `vitals_data/replay.py` with a `Replayer` class + CLI; a test that replays a short run at high speed and asserts the expected number of windows were emitted in order.

ACCEPTANCE: ``make replay`` streams windows the edge service can consume; fault toggle works.

C Machine-Learning Models

PROMPT 6 — Anomaly detector (unsupervised)

→ [chat](#) or [Claude Code](#)

CONTEXT: First line of defence - trained only on HEALTHY data, flags anything abnormal.

TASK: Train an autoencoder (PyTorch) on healthy windows; use reconstruction error as the anomaly score. Also fit an Isolation Forest baseline for comparison. Select the alert threshold from the healthy-error distribution (e.g. 99th percentile) and save it as an artifact.

REQUIREMENTS: use the shared preprocess module; log train/val loss; save model + threshold + scaler together as one versioned artifact folder.

DELIVERABLES: `vitals_train/train_anomaly.py`; a metrics printout (ROC-AUC on held-out healthy-vs-fault); saved artifact.

ACCEPTANCE: reconstruction error clearly separates healthy from seeded/known-fault windows; artifact reloads and scores a single window.

PROMPT 7 — RUL model - CNN-LSTM (supervised)

→ chat or Claude Code

CONTEXT: Turns degradation trend into cycles-to-failure. C-MAPSS FD001 is the benchmark.
TASK: Implement a compact CNN-LSTM: 1-D conv feature extractor -> LSTM -> linear RUL output. Build the sliding-window Dataset, training loop with early stopping, and evaluation using BOTH RMSE and the standard C-MAPSS asymmetric scoring function (late predictions penalised more). Clip RUL labels with the usual piece-wise-linear cap.
REQUIREMENTS: MSE (or asymmetric) loss; reproducible seed; checkpoint best model; keep params small enough for INT8 later; report metrics vs a plain-LSTM baseline.
DELIVERABLES: vitals_train/models.py (model defs), vitals_train/train_rul.py, a metrics table.
ACCEPTANCE: RMSE in the competitive range (~15-20 cycles on FD001); best checkpoint saved.

PROMPT 8 — Export & quantize for the edge

→ chat or Claude Code

CONTEXT: Edge inference must be fast and light.
TASK: Export both trained models to ONNX; add an optional TensorRT INT8 build path for Jetson. Write a parity check comparing PyTorch vs ONNX outputs on the same inputs, and a micro-benchmark printing p50/p95 inference latency.
REQUIREMENTS: graceful fallback to ONNX Runtime (CPU) when TensorRT / CUDA is absent, so it runs on a laptop; keep a single ModelRunner interface for both backends.
DELIVERABLES: vitals_train/export.py + vitals_edge/runner.py (ModelRunner) + a latency report.
ACCEPTANCE: ONNX output matches PyTorch within tolerance; ModelRunner scores a window on CPU.

D Edge Service (the core)

PROMPT 9 — Inference core

→ chat or Claude Code

CONTEXT: Wraps both models behind one call the service can use.
TASK: Build an InferenceCore that loads the anomaly + RUL artifacts via ModelRunner, applies the shared preprocessing to an incoming SensorWindow, and returns a Prediction (anomaly_score, rul_estimate, fault_class, confidence).
REQUIREMENTS: load artifacts once at startup; thread-safe scoring; no preprocessing code duplicated (import vitals_common.preprocess).
DELIVERABLES: vitals_edge/inference.py + a test scoring a replayed window end-to-end.
ACCEPTANCE: given a SensorWindow, returns a valid Prediction in <300 ms on CPU.

PROMPT 10 — Decision engine (state machine)

→ chat or Claude Code

CONTEXT: Raw scores must become a stable, actionable health state.
TASK: Implement a decision engine mapping (anomaly_score, rul_estimate, thresholds) to a state HEALTHY/WATCH/WARNING/CRITICAL, with hysteresis / debounce so the state does not flap on noise, plus a fault_class label when the pattern is known.
REQUIREMENTS: thresholds from config; hysteresis window configurable; pure and unit-testable; emit a HealthEvent only on state change or on a heartbeat interval.
DELIVERABLES: vitals_edge/decision.py + tests feeding a scripted score sequence and asserting the exact state transitions and that no flapping occurs.
ACCEPTANCE: a rising anomaly trend produces HEALTHY->WATCH->WARNING->CRITICAL without oscillation.

PROMPT 11 — FastAPI edge service

→ [Claude Code](#)

CONTEXT: The on-device app that ties everything together. Reuses our FastAPI experience.

TASK: Build a FastAPI service exposing GET /health (liveness + model versions), POST /predict (a SensorWindow -> Prediction + state), and GET /alerts (recent events). On each incoming window it runs InferenceCore -> decision engine -> publishes an Alert over MQTT and triggers the local alert.

REQUIREMENTS: pydantic request/response models from vitals_common; startup loads artifacts; structured logging of latency per request; config-driven; runs with `uvicorn`.

DELIVERABLES: vitals_edge/app.py + an httpx-based test hitting all three endpoints.

ACCEPTANCE: `make run-edge` serves; POST /predict returns state; an MQTT alert is published.

PROMPT 12 — Ingest adapters

→ [chat](#) or [Claude Code](#)

CONTEXT: The service must accept data from real sensors OR the replay harness through one path.

TASK: Define an Ingest interface with implementations: ReplayIngest (from Prompt 5), MqttIngest (subscribe to a sensor topic), and a stubbed CanIngest reading OBD-II PIDs via python-can / SocketCAN (guarded so it no-ops off-Linux). Each yields SensorWindow objects to the service.

REQUIREMENTS: common async interface; selectable via config; CanIngest import-guarded so laptop runs never fail.

DELIVERABLES: vitals_edge/ingest.py + a test running ReplayIngest into the service loop.

ACCEPTANCE: switching INGEST=replay|mqtt|can in .env changes the source with no code edits.

PROMPT 13 — Local alert (connectivity-independent)

→ [chat](#) or [Claude Code](#)

CONTEXT: The safety signal must fire even with no network - the heart of the "edge" claim.

TASK: Implement a local alerter that drives a GPIO LED/buzzer on Jetson/Pi when state >= WARNING, with a laptop fallback that prints a coloured console banner and (optionally) a desktop notification.

REQUIREMENTS: hardware access import-guarded; single Alerter interface; debounced to match the decision engine.

DELIVERABLES: vitals_edge/alert.py + a test asserting the fallback fires on a CRITICAL event.

ACCEPTANCE: on a laptop, a WARNING/CRITICAL state produces a visible local alert with no network.

E Thin Cloud Layer

PROMPT 14 — MQTT subscriber + storage

→ [chat](#) or [Claude Code](#)

CONTEXT: The cloud only ever receives small alert packets.

TASK: Build a subscriber that connects to Mosquitto, validates incoming Alert / HealthEvent JSON

against the shared schemas, and writes to InfluxDB (or SQLite for the demo) with a sane retention

policy and an asset_id index.

REQUIREMENTS: reconnect on drop; buffer to disk if the DB is unavailable; config-driven; a docker

-compose snippet for Mosquitto + InfluxDB.

DELIVERABLES: vitals_cloud/subscriber.py, storage.py, docker-compose.yml + a test using an embedded/local broker or a mock.

ACCEPTANCE: a published Alert lands in the store and is queryable by asset_id and time.

PROMPT 15 — Lightweight digital twin

→ [chat](#) or [Claude Code](#)

CONTEXT: A small model that projects the degradation curve forward for service planning.

TASK: From an asset's recent RUL/health history, fit a simple degradation trend (linear or exponential) and project the estimated failure date + a confidence band; expose a what_if(load_factor) that shifts the curve for a heavier/lighter duty cycle.

REQUIREMENTS: no heavy 3D engine - pure numpy/scipy; returns data the dashboard can plot; handles

sparse history gracefully.

DELIVERABLES: vitals_cloud/twin.py + a test on a synthetic degradation series.

ACCEPTANCE: projected failure date moves sensibly when load_factor changes.

PROMPT 16 — Streamlit dashboard

→ [Claude Code](#)

CONTEXT: What the judges look at while the edge box does the work.

TASK: Build a Streamlit dashboard with: live health tiles per asset (colour by state), an RUL countdown chart, the digital-twin projection, a scrolling event log, and a simple fleet view. It reads from the store (Prompt 14) and/or subscribes to MQTT for live updates.

REQUIREMENTS: auto-refresh; degrades gracefully if the edge feed stops (shows "last seen"); clean, demo-ready styling.

DELIVERABLES: vitals_cloud/dashboard.py + `make run-dashboard`.

ACCEPTANCE: with replay + edge running, the dashboard updates in real time and the RUL curve visibly counts down.

F Testing, Demo & Docs

PROMPT 17 — End-to-end integration + latency harness

→ [Claude Code](#)

CONTEXT: Prove the whole chain works and meets the <2 s KPI.
TASK: Write an integration test / script that starts a local broker, launches the edge service, runs the replay harness (with fault), subscribes as the cloud, and asserts: an alert is produced, the state reaches WARNING/CRITICAL after the fault, and end-to-end latency (window -> alert) has p95 < 2 s. Print a small latency report (p50/p95/max).
REQUIREMENTS: runnable via `make test-e2e`; deterministic; tears down cleanly.
DELIVERABLES: tests/test_e2e.py + the latency report format.
ACCEPTANCE: e2e test passes and reports p95 < 2 s on a laptop.

PROMPT 18 — Offline-resilience test

→ [chat](#) or [Claude Code](#)

CONTEXT: The signature differentiator - it keeps working with no network.
TASK: Add a test that runs the edge service + replay, then simulates network loss (broker unreachable). Assert that: local detection still runs, the local alert still fires, alerts are buffered, and on reconnect the buffered alerts flush to the cloud in order.
DELIVERABLES: tests/test_offline.py + any buffering code needed in the edge alert publisher.
ACCEPTANCE: with the broker down, local WARNING/CRITICAL still fires; buffered alerts resync later.

PROMPT 19 — Demo automation & seeded fault

→ [Claude Code](#)

CONTEXT: A one-command, repeatable demo for the live round.
TASK: Create a demo script that boots everything, starts a HEALTHY replay, then on a keypress (or timer) injects the fault so the state visibly climbs and the local alert fires; include a printed "pull the network cable now" cue and confirm local detection continues.
REQUIREMENTS: single entrypoint (`make demo`); resilient to restart; on-screen narration of what is happening.
DELIVERABLES: scripts/demo.py + a short DEMO.md storyboard with timings.
ACCEPTANCE: `make demo` runs the full healthy->fault->alert->offline story end to end.

PROMPT 20 — Docs & pitch assets

→ [chat](#) or [Claude Code](#)

CONTEXT: Round-1 and round-2 need clear artefacts.
TASK: Generate a polished README (problem, architecture, quickstart, results), a Mermaid architecture diagram, an auto-generated metrics report (pulls the latest model metrics), and a 2-minute demo script mapped to the three judging stages.
REQUIREMENTS: everything reproducible from the repo; numbers pulled from artifacts, not hardcoded.
DELIVERABLES: README.md, docs/architecture.mmd, docs/metrics_report.md, docs/pitch.md.
ACCEPTANCE: a newcomer can go from clone to running demo using only the README.

G Reusable Meta-Prompts

Keep these four handy - you'll use them across every card above.

PROMPT 21 — Plan-before-code

→ any assistant

Before writing any code for [TASK], give me: (1) the files you will create or change, (2) the key functions/classes and their signatures, (3) any assumptions or decisions that are hard to reverse, and (4) how you will test it. Wait for my "go" before writing the full implementation.

PROMPT 22 — Code review

→ any assistant

Review the code below as a senior engineer for VITALS-Edge. Check: correctness, train/serve skew (is preprocessing shared?), edge performance (will it quantize / stay <300 ms?), error handling, missing tests, and anything that would embarrass us in a live demo. List issues by severity (blocker / should-fix / nit) and show the diff for the blockers only. CODE: [paste]

PROMPT 23 — Debug

→ any assistant

This is failing. Here is the error, the command I ran, and the relevant code. Give me the most likely root cause first (one sentence), then the fix as a diff, then a one-line test that would have caught it. Do not rewrite unrelated code. ERROR: [paste] COMMAND: [paste] CODE: [paste]

PROMPT 24 — Refactor for the edge

→ any assistant

Refactor [MODULE] so it runs efficiently on a Jetson Orin Nano Super and still on a laptop CPU: reduce allocations in the hot path, keep the ModelRunner interface, ensure the model stays INT8-quantizable, and add a micro-benchmark. Keep behaviour identical and prove it with the existing tests. MODULE: [paste path]

Suggested Build Order

Follow the cards top to bottom - they mirror the build pipeline in the dossier (Acquire → Train → Export → Replay → Edge inference → Alert + dashboard). A practical two-week rhythm:

- **Days 1–2:** Prompts 0–5 — repo, schemas, data, preprocessing, replay. You can already stream windows.
- **Days 3–6:** Prompts 6–8 — train anomaly + RUL, export. You now have real metrics for round 2.
- **Days 7–9:** Prompts 9–13 — the edge service, decision engine, ingest, local alert. Sub-2 s on device.
- **Days 10–11:** Prompts 14–16 — thin cloud + dashboard. The visual story is live.
- **Days 12–14:** Prompts 17–20 — e2e + offline tests, demo automation, docs. Rehearse the network-cut moment.

Meta-prompts 21–24 are used continuously throughout. Adjust bracketed placeholders to your actual paths and numbers as the repo grows.